

A Cross-model Fusion-aware Framework for Optimizing (gather-matmul-scatter)_s Workload

Yaoxiu Lian¹, Zhihong Gou¹, Yibo Han¹, Zhongming Yu⁵,
Jiaming Xu¹, Sheng Yuan², Zhilin Pei², Xingcheng Zhang², Ningyi Xu¹, Guohao Dai^{1,3,4†}

¹Shanghai Jiao Tong University

²Shanghai Artificial Intelligence Laboratory Independent Researcher

³Infinigence-AI ⁴SII ⁵Independent Researcher

[†]Corresponding author: daiguohao@sjtu.edu.cn

Abstract—Modern deep learning models, such as Relation Graph Convolutional Network (RGCN), Sparse Convolutional Networks (SpConv), and Mixture of Experts Networks (MoE), are significantly dependent on the (gather-matmul-scatter)_s (abbreviated as (g-mm-s)_s) workload as their fundamental computational pattern. While existing works have made optimization attempts, several critical challenges remain unsolved, including domain-specific optimization migration, time-consuming exploration, and inefficient dataflow with dynamic inputs.

To address these challenges, we introduce **Efficient-GMS**, a comprehensive framework that enhances (g-mm-s)_s workload across diverse input scenarios. Our framework introduces (1) A Fusion-aware framework enabling cross-model optimization migration. We propose a comprehensive dataflow analysis that identifies shared computational patterns across models, enabling the development of four optimized dataflow patterns with vertical and horizontal fusion strategies. (2) Performance model-guided configuration space reduction. We develop a performance model to predict the relative execution efficiency across configurations, thereby reducing the search space and minimizing search time while ensuring optimal configuration selection. (3) Adaptive dataflow selection mechanism. We implement a lightweight heuristic model that dynamically selects optimal dataflow patterns based on the characteristics of the input and the hardware. Experimental results demonstrate that Efficient-GMS achieves significant performance gains, delivering an average end-to-end speedup of $1.46\times$ in RGCN model, $1.32\times$ in SpConv-based model, and $1.15\times$ in MoE model compared to state-of-the-art methods.

Index Terms—gather, scatter, matmul

I. INTRODUCTION

The advancement of deep learning models has enabled us to address complex relationships and patterns in irregular data. Examples include Relational Graph Convolutional Networks (RGCN [19], [29], [31]), Sparse Convolutional Networks (SpConv [12], [23]–[25]), and Mixture of Experts (MoE [7], [15]). These models, which support a wide range of applications from social network analysis [10], [13], [16], [26] to natural language processing [5], [14], [21], share a core computational pattern: (gather-matmul-scatter)_s (abbreviated as (g-mm-s)_s), where the subscript “s” indicates the plural form.

The (g-mm-s)_s workload represents a unified abstraction of core computations across these models. It consists of three key operations: (1) “gather”, which collects features from relevant sources (e.g., neighboring nodes in RGCN, non-zero elements in SpConv, or expert components in MoE); (2) “matmul”, which transforms these features through learnable weight matrices; and (3) “scatter”, which distributes the updated

results to appropriate destinations. This pattern, though conceptually straightforward, presents significant implementation challenges due to its varied forms across different models.

Although (g-mm-s)_s captures the core computations of these models, traditional implementation methods often struggle to efficiently execute (g-mm-s)_s in models with irregular structures, such as graphs with multiple relationships. For example, in RGCN, when PyG [8] executes the computation flow, it processes each set of gather, matmul, and scatter operations sequentially. This approach is inefficient due to indirect memory access and the challenges of fully utilizing parallel processing resources. Some studies [6], [22], [32], focus on optimizing multiple matmuls within the workload; however, they neglect the inter-operator relationships across the entire workload. Consequently, the performance improvements of the optimized models are limited.

Based on our analysis, we identify three fundamental challenges in optimizing (g-mm-s)_s:

- **Domain-specific optimization hindering cross-model migration.** Current optimization techniques are often designed for specific models, and applying these strategies to different models can result in significant performance differences. In some cases, directly using these techniques can cause severe performance drops—for example, applying the SpConv strategy to an RGCN model on the BGS dataset results in a $9.61\times$ decrease in performance.
- **Time-consuming exploration within large configuration space.** The configuration space for operators within (g-mm-s)_s workload is extensive, often encompassing thousands to even tens of thousands of possible configurations. Traditional approaches demand significant search time (up to 3000 seconds) to identify the optimal configuration. Furthermore, directly specifying configurations based on heuristic rule may result in suboptimal performance outcomes.
- **Inefficient static dataflow with dynamic inputs.** Traditional static optimization methods struggle with dynamic inputs, leading to inefficiencies with varying data characteristics. Performance varies widely across dataflow patterns, yet current systems lack the ability to dynamically select the most efficient pattern. Our experiments show that poor dataflow selection can cause up to a $14.77\times$ performance degradation under varying workloads.

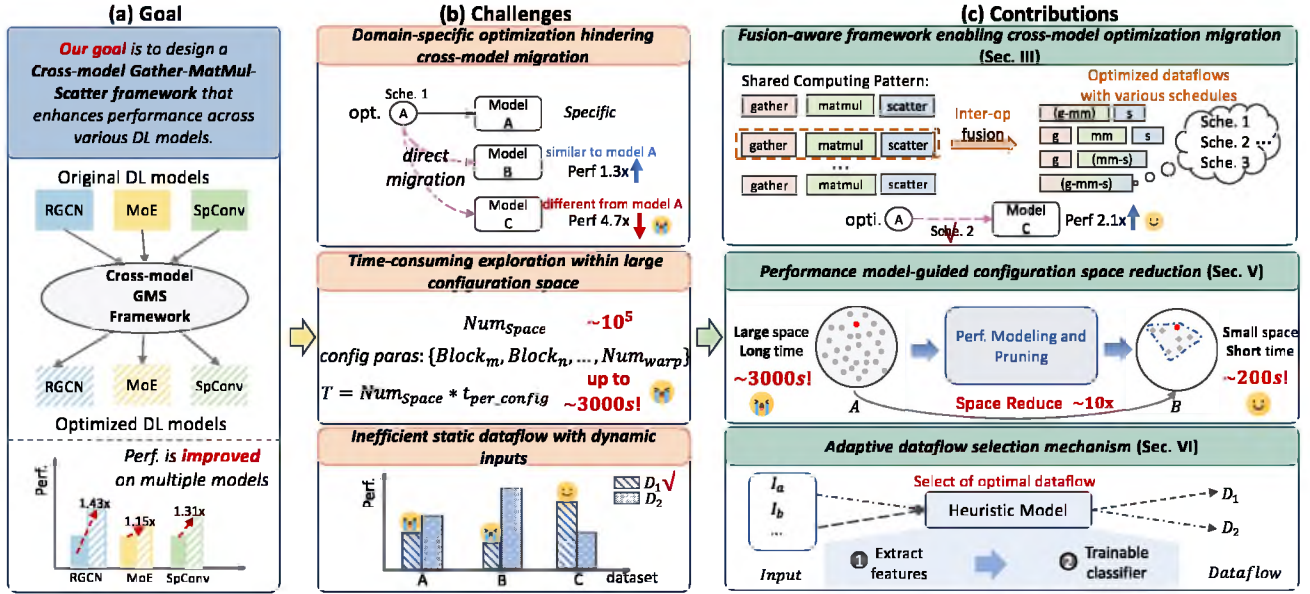


Fig. 1. Overview of Efficient-GMS, cross-model gather-matmul-scatter framework that enhances performance across various DL models.

To address these challenges, we present an efficient $(g\text{-mm-s})_s$ framework with the following key contributions:

- **Fusion-aware framework enabling cross-model optimization migration.** We develop a comprehensive optimization framework that identifies fundamental computational workload shared by different models. By analyzing the commonalities of workloads, we present four optimized dataflow patterns that combine fusion strategies and implement optimal scheduling strategies for different inputs. This approach achieves stable performance improvements from $1.43\times$ to $1.86\times$ in different models.
- **Performance model-guided configuration space reduction.** We develop a performance model based on parallel efficiency and the compute-to-memory access ratio that predicts the relative execution efficiency for each configuration, significantly reduces the search space by 60% to 90% while ensuring optimal configuration selection, and achieves a $5.26\times$ reduction in average search time.
- **Adaptive dataflow selection mechanism.** We propose a lightweight heuristic model that dynamically selects optimal dataflow patterns based on input and hardware characteristics. This approach delivers $1.67\times$ to $6.03\times$ performance improvement compared to static implementations, effectively addressing the dynamic input challenge.

In conclusion, we introduce Efficient-GMS, a comprehensive framework designed to enhance $(g\text{-mm-s})_s$ workload across models and tasks. Experimental results demonstrate that our approach achieves an average speedup of $1.73\times$ and $1.52\times$ on the $(g\text{-mm-s})_s$ kernel for RGCN and SpConv-based model, respectively. Furthermore, the end-to-end acceleration effects are significant, with average speedups of $1.43\times$, $1.31\times$, and $1.15\times$ on RGCN, SpConv-based model, and MoE tasks, respectively.

II. BACKGROUND

Relational Graph Convolutional Networks (RGCN), Sparse Convolutional Networks (SpConv), and Mixture of Expert Models (MoE) have emerged as sophisticated computational frameworks for tackling complex tasks in diverse domains. RGCNs excel at capturing relationships in graph-structured data [17], [18], SpConvs efficiently process sparse data like 3D point clouds [1], [27], and MoEs leverage specialized sub-networks to handle different input patterns [14], [30]. While seemingly disparate, these models share an underlying computational theme: they all adaptively aggregate information from a selective subset of the input. In this section, we introduce these three models, focus on their dataflow and demonstrating their adherence to the $(g\text{-mm-s})_s$ workload.

A. Relational Graph Convolutional Network (RGCN)

RGCN [19] extends the gather-matmul-scatter pattern to handle multi-relational graph data. During gathering, it collects information from neighboring nodes based on the graph's connectivity, considering different edge types as distinct relations. The matmul stage applies relation-specific weight matrices to transform the gathered information, enabling the network to learn relation-dependent feature representations. The scatter stage combines transformed information across all relations through summation, producing updated node representations that capture the graph's relational structure.

B. Sparse Convolutional Network

Sparse-based networks [12] implement an efficient gather-matmul-scatter pattern for sparse data like 3D point clouds. The gather stage collects features from neighboring points within a geometrically defined receptive field (e.g., sphere or cube). The matmul stage applies learned filters to gathered features, performing convolution in sparse space while computing only active points. The scatter stage aggregates these convolved features through summation, updating the

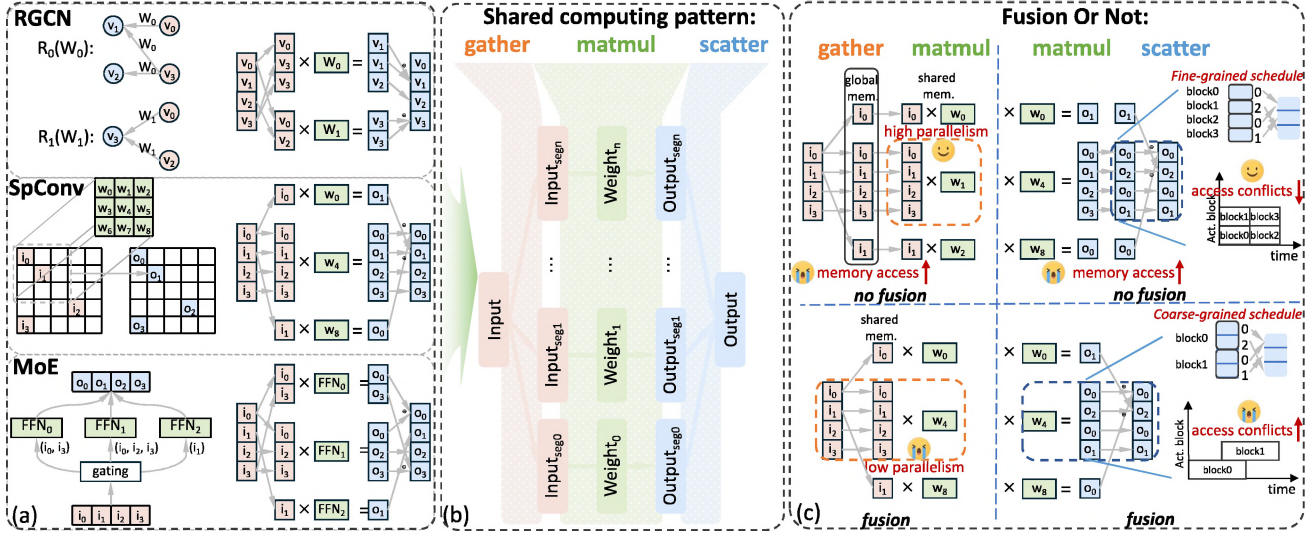


Fig. 2. (a). We demonstrate corresponding dataflows in RGCN, SpConv and MoE model. (b). gather-matmul-scatter is the shared computing pattern among three models. (c). Analyzing the advantages and disadvantages of fusion between operators.

central point's representation while maintaining computational efficiency through sparsity exploitation.

C. Mixture of Expert Model

MoE architectures [7], [15] adapt the gather-matmul-scatter pattern to ensemble learning. The gather stage employs a gating network that routes inputs to appropriate expert subsets. In the matmul stage, selected experts process inputs in parallel, each applying specialized computations. The scatter stage combines expert outputs through gating-weighted aggregation, producing final predictions that leverage multiple specialized networks. This architecture enables efficient scaling by activating only relevant experts for each input.

III. FUSION-AWARE FRAMEWORK ENABLING CROSS-MODEL OPTIMIZATION MIGRATION

A. Motivation

The optimization techniques developed for various deep learning models demonstrate notable potential for cross-model applicability. For instance, the gather-matmul-scatter fusion used in SpConv can be adapted to RGCN's feature aggregation, while RGCN's segmentation-based computation strategy can benefit expert computation in Mixture of Experts (MoE). Experiments demonstrate notable performance gains from such migrations; for example, RGCN's feature aggregation on the IMDB dataset achieves a $4.64\times$ speedup using SpConv's strategy.

B. Challenge

Despite the variety of optimization methods, most implementations remain domain-specific. Direct migration often leads to instability due to differences in model implementation details and data performance characteristics. For example, applying the operator fusion strategy to RGCN on the BGS dataset results in a $9.61\times$ performance drop.

C. Insight

Our analysis reveals that seemingly diverse models share a fundamental (g-mm-s)_s computational workload. This

commonality enables the construction of a unified optimization framework, dynamically guiding strategy migration across models based on operator and data characteristics. Experiments validate the framework's efficacy, achieving stable performance improvements between $1.15\times$ and $1.46\times$ across various models.

D. Approach

We propose a **systematic optimization framework for (g-mm-s)_s workload** and thoroughly analyze potential optimization methods in both vertical and horizontal dimensions. **Vertically**, parallel operator scheduling focuses on enhancing hardware utilization by combining small-scale gather, matmul, or scatter operations into larger ones. Inspired by segment-matmul methods [32], parallel scheduling of fused matmul operations significantly boosts performance. **Horizontally**, inter-operator fusion analyzes the interrelationships among gather, matmul, and scatter to refine four fundamental dataflows: $G - M_{seg} - S$, executing operators independently; $(GM)_{seg} - S$, combining gather and matmul; $G - (MS)_{seg}$, merging matmul and scatter; and $(GMS)_{seg}$, fully integrating all three operators. At the same time, the above four dataflows are executed based on guaranteeing parallel operator scheduling, and the subscript in the dataflow represents that the operator organizes operations in the way of the segment. These dataflows provide a versatile optimization framework adaptable to varying workloads and hardware architectures. Fusion can enhance data locality and reduce overhead, though it may increase memory access conflicts, affecting efficiency. By evaluating hardware characteristics, input data, and computational workload, the framework enables the dynamic selection of optimal dataflow, achieving significant performance gains across diverse computational scenarios.

E. Op Fusion Analysis

The fusion of gather and matmul operators involves a trade-off, especially for large-scale computations. Independent operators often outperform fused implementations by leveraging

enhanced parallelism. Separating gather operations enables concurrent processing across thread blocks, decoupling irregular memory access from structured matrix multiplication and maximizing GPU parallelism. This design also allows dynamic resource allocation and avoids bottlenecks seen in fused implementations.

For the fused $(MS)_{seg}$ kernel, performance suffers from atomic write conflicts during scatter operations. While fusion improves data reuse and reduces memory accesses, handling more rows per block increases conflicts, extending execution time. Separating matmul and scatter mitigates these conflicts, offering greater configurability and improved efficiency.

IV. PERFORMANCE MODEL-GUIDED CONFIGURATION SPACE REDUCTION

A. Motivation

The selection of configuration parameters in the dataflow directly determines computational efficiency, making configuration space optimization crucial for performance enhancement. In the $(g\text{-}mm\text{-}s)_s$ workload, gather operations involve non-contiguous memory access, matmul requires effective resource scheduling, and scatter entails complex data distribution. Parameters such as block size, affecting data locality, and thread allocation, influencing parallelism, are critical. Suboptimal configurations can degrade memory access efficiency and under-utilize computational resources.

B. Challenge

The optimization of configuration space is challenging due to its high dimensionality and interdependent parameters. In the $(g\text{-}mm\text{-}s)_s$ pattern, parameters like tile sizes ($Block_M$, $Block_N$, $Block_K$) influence data partitioning, while thread counts (Num_{warp}) control parallelism. These parameters are interrelated: larger tiles improve data reuse but limit parallelism, while more threads increase parallelism but risk resource contention. Input variations, such as sparsity and data scale, further complicate configuration needs, expanding the space to tens of thousands of possibilities. Exhaustive searches, taking thousands of seconds, are thus impractical.

C. Insight

Our key insight is that **configuration parameters and performance indicators can be quantitatively characterized**, forming a foundation for efficient optimization. Configuration parameters primarily impact performance through parallelism and memory access efficiency: tile size influences data locality and cache usage, while thread allocation affects resource utilization. These relationships are predictable and quantifiable, enabling theoretical modeling to evaluate configuration effects without execution. This facilitates the rapid identification of optimal configurations.

D. Approach

To address these challenges, we propose a novel approach for **optimization configuration through coarse-grained performance modeling**. As shown in Fig. 3(b), The proposed model evaluates the relative execution efficiency under various configurations while keeping the computational overhead

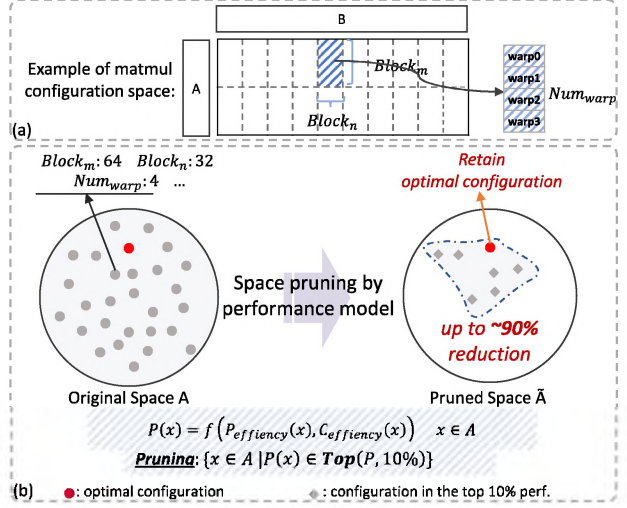


Fig. 3. (a). Example configuration space for matmul, which explains the meaning of the three configurable parameters (b). Performance-guided configuration space reduction.

minimal by analyzing the parallel efficiency and compute-to-memory efficiency of the operators under the current input. Subsequently, configurations within space A are ranked and pruned to generate a reduced configuration space B , comprising approximately 10% to 40% of the original space. The system then executes configurations exclusively within space B on the kernel, selecting the configuration with optimal average execution time, thus substantially reducing the optimization process duration.

Taking the $(GM)_{seg}$ operator as an example, its performance can be modeled as follows:

$$\begin{aligned}
 P &= (1 - \text{Pall}_{efficiency}) \cdot (1 - \text{Comp}_{efficiency}) \\
 \text{Comp}_{efficiency} &= T_{comp} / (T_{load} + T_{store} + T_{comp}) \\
 \text{Pall}_{efficiency} &= N_{blk} / ([N_{blk} / N_{max_act_blk}] \cdot N_{max_act_blk}) \\
 N_{blk/sm} &= \min(\text{shm}_{max} / \text{shm}_{required}, N_{thread/sm} / (\text{warp} \cdot 32)) \\
 N_{max_act_blk} &= N_{blk/sm} \cdot N_{sm} \\
 T_{comp} &= 2MNK / \text{PeakFLOPS} \\
 T_{store} &= (\text{st_bytes/sm} \cdot N_{max_act_blk} \cdot [N_{blk} / N_{max_act_blk}]) / L2_bw \\
 T_{load} &= T_{load_A} + T_{load_B} \\
 T_{load_A/B} &= \frac{\text{bytes}_{A/B}}{(1 - L2_hit_{A/B}) \cdot \text{dram_bw} + L2_bw \cdot L2_hit_{A/B}}
 \end{aligned}$$

The modeling process for other operators is similar to that of $(GM)_{seg}$. Additionally, we utilize a caching mechanism to associate optimal configurations with their corresponding data characteristics. This enables direct retrieval of optimal configurations when encountering similar data characteristics, eliminating the need for redundant search processes.

V. ADAPTIVE DATAFLOW SELECTION MECHANISM

A. Motivation

Dynamically selecting the optimal dataflow pattern is essential to optimize the performance of the $(g\text{-}mm\text{-}s)_s$ framework. Experimental results demonstrate that no single dataflow pattern consistently achieves optimal performance across diverse input characteristics. This performance variance is primarily driven by the dynamic changes in input data attributes, such as data scale, sparsity, and access patterns,

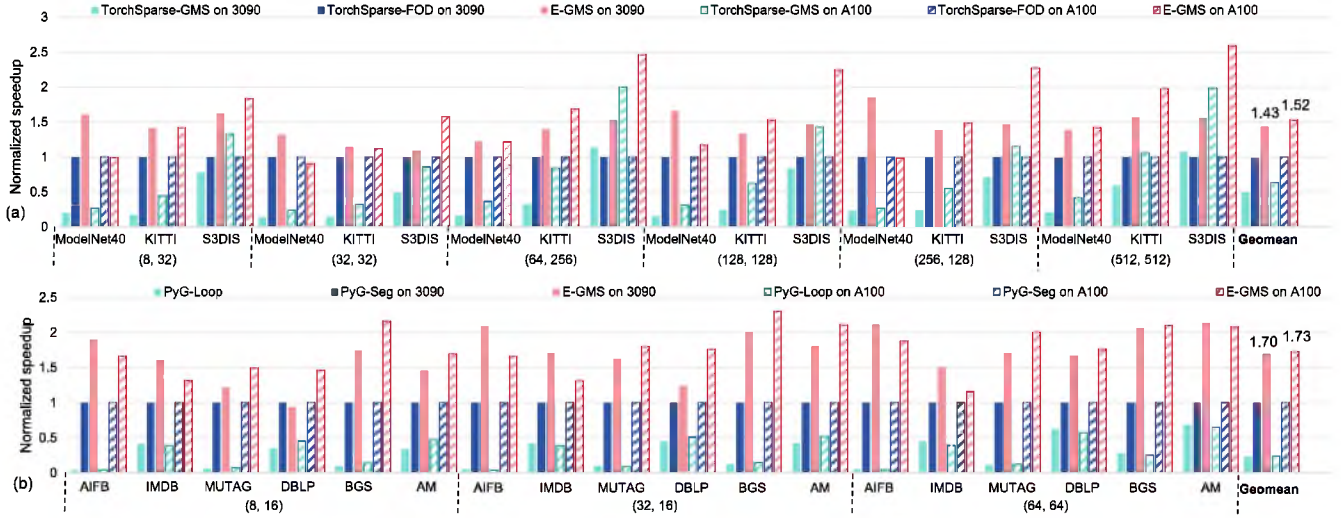


Fig. 4. Kernel performance. (a) kernel performance comparison between E-GMS and Torchsparse on the point cloud datasets. (b) kernel performance comparison between E-GMS and PyG on the heterogeneous graphs.

all of which significantly affect the execution efficiency of dataflow. Consequently, it is crucial to design a mechanism that dynamically selects the most suitable dataflow based on the specific input characteristics.

B. Challenge

Traditional static optimization strategies are not efficient in dynamic scenarios. Existing methods typically rely on predefined heuristic rules or fixed optimization approaches, which make it difficult to adjust the optimization strategy in time when facing input data features, leading to significant performance fluctuations. As shown in Fig. 8, selecting a static dataflow pattern can result in performance degradation of up to $14.77\times$.

C. Insight

A critical insight reveals that **dataflows exhibit different relative performances on different data**. Our analysis reveals two critical characteristics of dataflow performance: (1) the $(GM)_{seg} - S$ and $(GMS)_{seg}$ dataflows outperform other patterns in most cases, collectively accounting for 96.50% of optimal cases (52.50% and 44.00%, respectively); and (2) while $G - M_{seg} - S$ or $G - (MS)_{seg}$ may achieve optimal performance in rare scenarios (2.75% and 0.75% of cases, respectively), their advantages are marginal, with a maximum performance gap of $1.06\times$. These findings simplify dataflow selection by focusing on $(GM)_{seg} - S$ and $(GMS)_{seg}$, ensuring effective performance optimization while reducing selection complexity.

D. Approach

To address dynamic dataflow selection in the $(g-mm-s)_s$ workload, we propose an **adaptive machine learning framework** leveraging **Singular Value Decomposition (SVD)** for dimensionality reduction and an **XGBoost classifier** for dataflow selection. Simple features, such as input-output dimensions, undergo standardization, while complex mapping features are processed using SVD to retain essential features. The framework merges these features into a unified vector,

preserving interpretability and core mapping information. The best data flow is predicted by a trained XGBoost classifier. Supporting $(GM)_{seg} - S$ and $(GMS)_{seg}$ modes, the framework dynamically selects efficient dataflow, enabling real-time performance optimization.

VI. EVALUATION

A. Experimental Setup

Our evaluation platform comprises NVIDIA GeForce RTX 3090 and A100 (80GB) GPUs, testing three key architectures: sparse point cloud models (SparseResNet [4]) with sparse convolution, heterogeneous graph neural networks (RGCN), and Mixture of Experts models (Mixture-8x7B). Comparisons include Torchsparse implementations with gather-matmul-scatter and fetch-on-demand dataflows for point cloud models, PyG implementations using for-loop and segment-matmul for RGCN and HuggingFace implementation for MoE models. We utilize standard datasets: KITTI [11], ModelNet40 [28], and S3DIS [2] for point clouds, and AIFB, MUTAG, BGS, AM [20], IMDB [3], DBLP [9] for heterogeneous graphs.

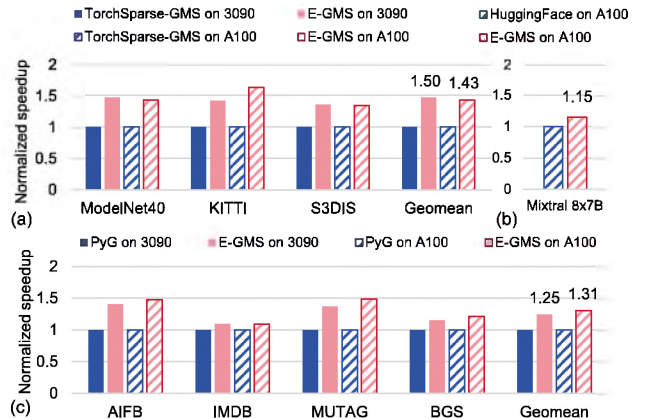


Fig. 5. End to end performance. (a) Performance comparison between E-GMS and Torchsparse on the point cloud datasets. (b) Performance comparison between E-GMS and PyG on the heterogeneous graphs.

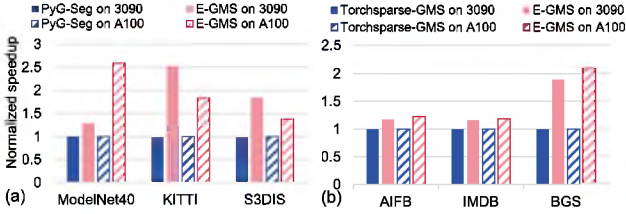


Fig. 6. Performance comparison between E-GMS and direct cross-model scheduling methods. (a) and (b) are the performance on SpConv-based and RGCN networks, respectively

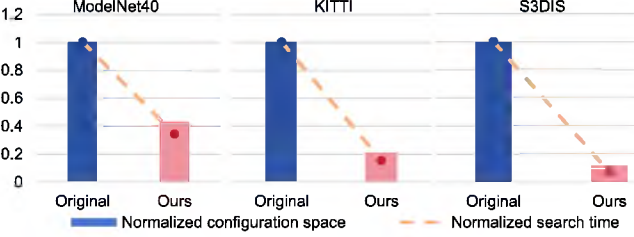


Fig. 7. Configuration space reduce and search time speedup

B. Overall Performance

End-to-end Performance. We evaluated the end-to-end performance of our proposed Efficient-GMS (E-GMS) method and the state-of-the-art baseline engines across three different scenarios, as shown in Fig. 5. The results show that on the SpConv-based models, E-GMS achieves an average speedup of $1.43\times$ and $1.46\times$ with respect to TorchSparse-FOD on RTX 3090 and A100, respectively. On RGCN, E-GMS achieves an average speedup of $1.25\times$ and $1.32\times$ on RTX 3090 and A100 compared to PyG, respectively. On MoE, E-GMS achieves an average speedup of $1.15\times$ compared to HuggingFace on A100. Notably, E-GMS achieves higher speedups over TorchSparse-FOD and PyG on A100 in two scenarios. This can be attributed to E-GMS’s superior GPU utilization, which results in higher performance gains on more powerful GPUs.

Kernel Performance. We further evaluate the kernel performance of GMS operations, showcasing the effectiveness of E-GMS by varying input and output channel sizes for each dataset. We set six and three different channel sizes (c_{in}, c_{out}) for point cloud and RGCN model, respectively. The results are shown in Fig. 4. E-GMS achieves an average arithmetic speedup of $1.44\times$ and $5.25\times$ over TorchSparse-FOD and TorchSparse-GMS using the segment-matmul method on RTX 3090, respectively. On A100, these values are $1.60\times$ and $2.58\times$, respectively. In particular, E-GMS achieves a peak speedup of $2.59\times$ over TorchSparse-FOD on large channel size (i.e., (512, 512) on A100), with $(GM)_{seg} - S$ being the optimal dataflow configuration. For the RGCN model, E-GMS achieves an average arithmetic speedup of $14.05\times$ and $1.7\times$ compared to PyG-Loop and PyG-Seg on RTX 3090, respectively. Similarly, on A100, E-GMS provides speedups of $13.78\times$ and $1.76\times$, showing its efficiency across different hardware platforms.

C. Ablation Study

Fusion-aware framework enabling cross-model optimization migration. As illustrated in Fig. 6, our approach outperforms the direct cross-model method, achieving an av-

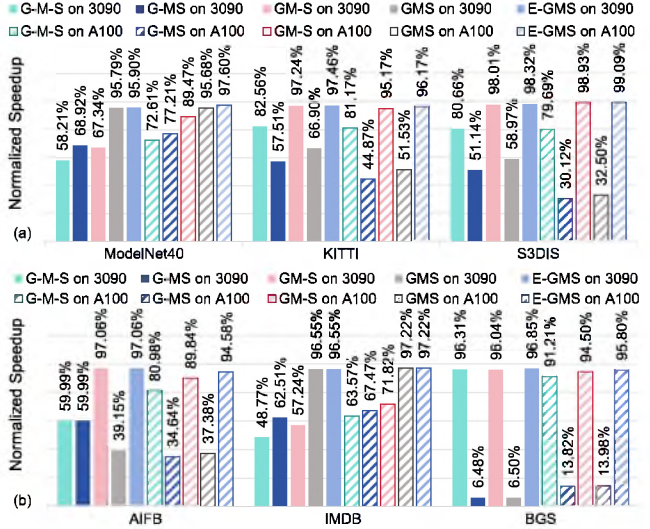


Fig. 8. Heuristic-model performance

erage performance improvement of $1.86\times$ on point cloud and a $1.43\times$ speedup on heterogeneous graphs on A100. These results demonstrate efficient scheduling of our method across diverse input types. **Performance model-guided configuration space reduction.** Fig. 7 highlights the efficiency of our performance model. For the GMS_{seg} operator on the point cloud, the performance model reduces the configuration space by 60%–90% and achieves an average search time reduction of $5.26\times$, with a notable $13.89\times$ reduction on large-scale datasets. **Heuristic Adaptive Dataflow.** Fig. 8 shows that the heuristic adaptive dataflow shows significant performance benefits, achieving $1.14\times$ to $1.67\times$ speedup on RTX 3090 and $1.03\times$ to $2.23\times$ on A100 for point cloud scenarios. RGCN scenarios show even more substantial improvements, with $1.23\times$ to $6.03\times$ speedup on RTX 3090 and $1.14\times$ to $3.70\times$ on A100, validating the effectiveness of our adaptive approach.

VII. CONCLUSION

In conclusion, this paper introduced Efficient-GMS, a comprehensive framework designed to enhance (g-mm-s)_s workload across models and tasks. By incorporating fusion-aware optimization, performance model-guided configuration selection, and an adaptive dataflow mechanism, Efficient-GMS successfully improves the workload under diverse input scenarios and hardware configurations. Our experimental results demonstrate that our approach achieves an average speedup of $1.73\times$ and $1.52\times$ on the (g-mm-s)_s kernel for RGCN and SpConv-based networks, respectively. Furthermore, the end-to-end acceleration effects are significant, with average speedups of $1.43\times$, $1.31\times$, and $1.15\times$ on RGCN, SpConv-based networks, and MoE tasks, respectively. We believe that the insights and techniques can benefit all tasks that fit in (g-mm-s)_s abstraction.

ACKNOWLEDGMENT

This work was sponsored by the National Natural Science Foundation of China (No. U21B2031), Shanghai Rising-Star Program (No. 24QB2706200).

REFERENCES

- [1] Iro Armeni, Ozan Sener, Amir R Zamir, Helen Jiang, Ioannis Brilakis, Martin Fischer, and Silvio Savarese. 3d semantic parsing of large-scale indoor spaces. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1534–1543, 2016.
- [2] Iro Armeni, Ozan Sener, Amir R Zamir, Helen Jiang, Ioannis Brilakis, Martin Fischer, and Silvio Savarese. 3d semantic parsing of large-scale indoor spaces. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1534–1543, 2016.
- [3] Yunsheng Bai, Hao Ding, Song Bian, Ting Chen, Yizhou Sun, and Wei Wang. Simgnn: A neural network approach to fast graph similarity computation. In *Proceedings of the twelfth ACM international conference on web search and data mining*, pages 384–392, 2019.
- [4] Christopher Choy, JunYoung Gwak, and Silvio Savarese. 4d spatio-temporal convnets: Minkowski convolutional neural networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 3075–3084, 2019.
- [5] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- [6] NVIDIA Corporation. Cutlass grouped kernel schedulers. https://github.com/NVIDIA/cutlass/blob/main/media/docs/grouped_scheduler.md. Accessed: 11-18-2024.
- [7] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- [8] Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with pytorch geometric. *arXiv preprint arXiv:1903.02428*, 2019.
- [9] Xinyu Fu, Jiani Zhang, Ziqiao Meng, and Irwin King. Magnn: Metapath aggregated graph neural network for heterogeneous graph embedding. In *Proceedings of the web conference 2020*, pages 2331–2341, 2020.
- [10] Thomas Gaudelet, Ben Day, Arian R Jamash, Jyothish Soman, Cristian Regep, Gertrude Liu, Jeremy BR Hayter, Richard Vickers, Charles Roberts, Jian Tang, et al. Utilizing graph machine learning within drug discovery and development. *Briefings in bioinformatics*, 22(6):bbab159, 2021.
- [11] Andreas Geiger, Philip Lenz, and Raquel Urtasun. Are we ready for autonomous driving? the kitti vision benchmark suite. In *2012 IEEE conference on computer vision and pattern recognition*, pages 3354–3361. IEEE, 2012.
- [12] Benjamin Graham and Laurens Van der Maaten. Submanifold sparse convolutional networks. *arXiv preprint arXiv:1706.01307*, 2017.
- [13] Yixin Guo, Pengcheng Li, Yingwei Luo, Xiaolin Wang, and Zhenlin Wang. Exploring gnn based program embedding technologies for binary related tasks. In *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension*, pages 366–377, 2022.
- [14] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multi-task language understanding. *arXiv preprint arXiv:2009.03300*, 2020.
- [15] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- [16] Yangyang Li, Yipeng Ji, Shaoning Li, Shulong He, Yinhao Cao, Yifeng Liu, Hong Liu, Xiong Li, Jun Shi, and Yangchao Yang. Relevance-aware anomalous users detection in social network via graph neural network. In *2021 international joint conference on neural networks (IJCNN)*, pages 1–8. IEEE, 2021.
- [17] Petar Ristoski, Gerben Klaas Dirk De Vries, and Heiko Paulheim. A collection of benchmark datasets for systematic evaluations of machine learning on the semantic web. In *The Semantic Web–ISWC 2016: 15th International Semantic Web Conference, Kobe, Japan, October 17–21, 2016, Proceedings, Part II 15*, pages 186–194. Springer, 2016.
- [18] Petar Ristoski and Heiko Paulheim. Rdf2vec: Rdf graph embeddings for data mining. In *International semantic web conference*, pages 498–514. Springer, 2016.
- [19] Michael Schlichtkrull, Thomas N Kipf, Peter Bloem, Rianne Van Den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *The semantic web: 15th international conference, ESWC 2018, Heraklion, Crete, Greece, June 3–7, 2018, proceedings 15*, pages 593–607. Springer, 2018.
- [20] Michael Schlichtkrull, Thomas N Kipf, Peter Bloem, Rianne Van Den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *The semantic web: 15th international conference, ESWC 2018, Heraklion, Crete, Greece, June 3–7, 2018, proceedings 15*, pages 593–607. Springer, 2018.
- [21] Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. Commonsenseqa: A question answering challenge targeting common-sense knowledge. *arXiv preprint arXiv:1811.00937*, 2018.
- [22] PyTorch Geometric Team. Pytorch geometric (pyg)-fastrgcnconv. https://github.com/pyg-team/pytorch_geometric/blob/master/torch_geometric/nn/conv/rgcn_conv.py. Accessed: 11-18-2024.
- [23] Xiaogang Wang, Marcelo H Ang, and Gim Hee Lee. Voxel-based network for shape completion by leveraging edge generation. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 13189–13198, 2021.
- [24] Yue Wang, Yongbin Sun, Ziwei Liu, Sanjay E Sarma, Michael M Bronstein, and Justin M Solomon. Dynamic graph cnn for learning on point clouds. *ACM Transactions on Graphics (tog)*, 38(5):1–12, 2019.
- [25] Xin Wen, Tianyang Li, Zhizhong Han, and Yu-Shen Liu. Point cloud completion by skip-attention network with hierarchical folding. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 1939–1948, 2020.
- [26] Shiwen Wu, Fei Sun, Wentao Zhang, Xu Xie, and Bin Cui. Graph neural networks in recommender systems: a survey. *ACM Computing Surveys*, 55(5):1–37, 2022.
- [27] Zhirong Wu, Shuran Song, Aditya Khosla, Fisher Yu, Linguang Zhang, Xiaoou Tang, and Jianxiong Xiao. 3d shapenets: A deep representation for volumetric shapes. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1912–1920, 2015.
- [28] Zhirong Wu, Shuran Song, Aditya Khosla, Fisher Yu, Linguang Zhang, Xiaoou Tang, and Jianxiong Xiao. 3d shapenets: A deep representation for volumetric shapes. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1912–1920, 2015.
- [29] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and S Yu Philip. A comprehensive survey on graph neural networks. *IEEE transactions on neural networks and learning systems*, 32(1):4–24, 2020.
- [30] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830*, 2019.
- [31] Jie Zhou, Ganqu Cui, Shengding Hu, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun. Graph neural networks: A review of methods and applications. *AI open*, 1:57–81, 2020.
- [32] Keren Zhou, Karthik Ganapathi Subramanian, Po-Hsun Lin, Matthias Fey, Binqian Yin, and Jiajia Li. Fasten: Fast gpu-accelerated segmented matrix multiplication for heterogeneous graph neural networks. In *Proceedings of the 38th ACM International Conference on Supercomputing*, pages 511–524, 2024.